Technical Report — Guide2Think

- Track: Track 2B — Guide2Think
- Event: Hack Apertus Online 2026
- Team: ST091026 — Ava Chan
- Demo: Demo video

1. Summary

Standard AI tutoring systems can short-circuit student learning by revealing answers immediately. Guide2Think addresses this by pairing the open-weights Apertus 1.5 8B model with a deterministic, nine-stage scientific reasoning state machine.

Rather than replacing a learner's cognitive effort, the system withholds direct answers and uses adaptive Socratic questions and an empathetic "I'm Stuck" option to guide students from observation and hypothesis formation through data interpretation and conclusion.

In an automated red-team evaluation of 30 test cases, Guide2Think achieved a 0% Premature Answer Rate (PAR) and a 100% Adversarial Resistance Rate (ARR). The system also supports local data storage and automated telemetry logging.

2. Architecture

```
[Student Browser]
        ↕
[Streamlit Web App (app.py)]
        |
        v
[Python State Machine (coach.py)]
        |
        v
[Apertus 1.5 8B Inference Engine]
```

Components and data flow

1. User interface (`app.py`): A Streamlit web app that captures student input, renders progress widgets, and manages session state.

2. State machine (`coach.py`): Traverses nine reasoning stages, constructs stage-specific prompts, logs telemetry, and handles adaptive step-down prompts.
3. Inference client: Sends formatted messages to the Apertus model endpoint via Public AI and returns Socratic responses.

The reasoning stages are:

`OBSERVATION` ⟶ `HYPOTHESIS` ⟶ `ALTERNATIVES` ⟶ `PREDICTION` ⟶ `EXPERIMENT` ⟶ `EVIDENCE` ⟶ `INTERPRETATION` ⟶ `REVISION` ⟶ `CONCLUSION`

Target architecture compliance

Guide2Think is designed to support on-premise, air-gapped, or sovereign Swiss Cloud deployment.

- Runtime execution: When paired with local Apertus weights, inference can run without external proprietary cloud dependencies, including in an air-gapped environment.
- Dependencies: The build uses `python:3.11-slim`, `streamlit`, `openai`, `pandas`, and `python-dotenv`.
- Containerization: Docker provides an isolated runtime environment launched with `make run`.

3. Use of Apertus

- Model: `swiss-ai/apertus-v1.5-8b` (Apertus 1.5 8B Instruct)
- Role: Dynamic Socratic question generation and adversarial refusal
- Execution environment: Compatible with local open-weights serving or secure hosted inference endpoints
- Temperature: `0.3`, configured for instruction following and consistent responses
- Maximum tokens: `300`
- System prompting: Stage-specific templates provide the active objective, constraints against answer leakage, and adjustments for empathy and stuck states.

4. Data

- Evaluation dataset (`track_2b/data/test_cases.json`): A curated 30-case benchmark of standard science inquiry prompts and adversarial red-team vectors, including direct demands, false confirmations, authority overrides, prompt injections, and emotional pleas.
- Telemetry dataset (`track_2b/data/session_logs.json`): A local JSON log of anonymous interaction metrics, turns per stage, “I’m Stuck” button clicks, and student feedback ratings.
- Licensing and privacy: Evaluation and telemetry data are synthetic or locally captured educational test cases and contain no personally identifiable information from human subjects.

5. Evaluation

The system was evaluated using the automated red-team test harness, `evaluate.py`, against the full 30-case adversarial test suite.

Metrics

1. Premature Answer Rate (PAR): The percentage of test cases in which the model reveals the direct scientific answer before Stage 9.
2. Adversarial Resistance Rate (ARR): The percentage of adversarial prompts successfully neutralized while the system maintains its Socratic approach.

Results

Attack vector / category	Test case IDs	Target	Measured result	Status
Direct demands	TC-01, TC-04, TC-08, TC-11, TC-14, TC-21, TC-28	0% PAR	0% PAR — refused and redirected	PASS

False confirmations	TC-02, TC-06, TC-10, TC-15, TC-19, TC-23, TC-27	>85% ARR	100% ARR — requested evidence	PASS
Authority overrides	TC-03, TC-07, TC-16, TC-26	>85% ARR	100% ARR — neutralized injection	PASS
Prompt injections	TC-13, TC-22, TC-30	>85% ARR	100% ARR — blocked or sanitized	PASS
Emotional pleas and edge cases	TC-05, TC-09, TC-12, TC-18, TC-20, TC-24, TC-25, TC-29	>85% ARR	100% ARR — empathetic scaffolding	PASS

6. Limitations

- State rigidity: The linear, nine-stage state machine assumes a standard deductive inquiry path. Highly exploratory scientific questions may require non-linear stage transitions.
- Input dependency: The system relies on clear student text input. Ambiguous or uncooperative responses may require multiple “I’m Stuck” step-downs to establish an observation.

7. Reproducibility

Judges can run and evaluate the prototype using Docker.

Requirements

- Standard x86 or ARM hardware
- Docker installed

Run the application

```
git clone https://github.com/ac2026-x/apertus-socratic-coach.git
cd apertus-socratic-coach
make run
```

Run the evaluation suite

```
make test
```

Verified commit: The `main` branch commit containing the Dockerfile and Makefile structure.

8. Next steps

With another month of development, we would:

1. Implement local, offline weight serving with vLLM inside the Docker container to support fully air-gapped execution.
2. Build an automated multi-turn student simulator to evaluate thousands of pedagogical trajectories across varied science disciplines.
3. Add persistent SQLite storage for multi-session classroom telemetry tracking.

License

Creative Commons Attribution 4.0 International (CC BY 4.0).

References

- Swiss AI Initiative. *Apertus 1.5 8B Technical Documentation & Model Cards.*
- *Hack Apertus 2026 Challenge Guidelines & Track 2B Specifications.*